



TECHNICAL SPESIFICATION

AI Trigger System

Abstract

The CNN Trigger System is a real-time, AI-based event selector designed to ingest high-speed raw data from front-end digitizers (ADC), perform low-latency inference using parallel CNN Accelerator Cores, and trigger the readout of high-fidelity waveform data.

The architecture employs a Split-Path Strategy to decouple data storage from analysis. Incoming data is bifurcated into a Storage Path (High-Fidelity Ground Truth) and an Inference Path (Downsampled/Selected Channels). A central PicoRV32 soft-core orchestrates a pool of parallel CNN Accelerators via a dedicated uDMA engine.



This template is intended for generic engineering documents only. Its use is subject to some recommendations. It must not be used for documents that are supporting specific processes such as Engineering Change Requests (ECR). Please see [all details](#) on the CERN ATS Quality Service website.

DOCUMENT PREPARED BY:

[Albert Chueng – UC Irvine]

DOCUMENT TO BE CHECKED BY:

TBD

DOCUMENT TO BE APPROVED BY:

TBD

DOCUMENT SENT FOR INFORMATION TO:

OPTIONAL: I am a recipient of information

SUMMARY OF THE ACTIONS TO BE UNDERTAKEN:

OPTIONAL: I am a summary



Contents

1 Introduction	6
1.1 Top-Level Block Diagram	6
1.2 Key Subsystems	6
2 Clocking and Synchronization	8
3 Functional Description	8
3.1 Split-Bank Memory Architecture	8
3.1.1 Ingress Signal Splitting (ingress_demux)	8
3.1.2 Banked Memory Roles	8
3.1.3 Addressing and Circularity	8
3.2 Task Orchestration	9
3.2.1 Dynamic “First-Available” Scheduler	9
3.3 Timing and Wrapper Logic	9
3.3.1 AI Accelerator Wrapper (ai_core_wrapper)	9
3.3.2 Two-Tier Timing Engine	10
3.4 Trigger and Re-Ordering Logic	10
3.4.1 Software Re-Order Buffer	10
3.4.2 Thresholding and Backend Data Move	11
4 CPU	11
4.1 Architectural Philosophy and Design Goals	12
4.2 Instruction Set Architecture (ISA) and Configuration	12
4.3 Performance Metrics	12
4.4 Native and System Interfaces	13
4.5 Custom Interrupt Handling and IRQ Management	13
4.5.1 Interrupt State Registers	13
4.5.2 Specialized IRQ Instructions	13
4.6 Built-in Interrupt Sources	13
5 Memory Map and Register Description	14
5.1 Global Control Registers	14
5.2 Core-Specific Registers	15
5.3 Hardware-Software Handshake Sequence	15
6 Timing and Performance Constraints	15
7 Operational Modes	16
7.1 Idle Mode	16
7.2 Run Mode	16
7.3 Passthrough Mode	16
7.4 Watchdog and Fault Recovery	17
8 System Initialization and Error Recovery	17
8.1 Cold Boot Sequence	17
8.2 Run Start Procedure	17
8.3 Watchdog-Driven Fault Recovery	18
A Appendix	19
A.1 Term and Parameter Definitions	19



List of Figures

1	Top Level Architecture of the AI Trigger System	6
---	---	---



List of Tables

1	Instruction Latency and CPI Performance	12
2	Global Control Registers Map	14
3	Core-Specific Registers (Offset from Core Base)	15
4	System Terms and Definitions	19



ARIANNA

REFERENCE
UCI-A-REF-0001

EDMS NO.	REV.	VALIDITY
-----	1.5	IN WORK



1 Introduction

The AI Trigger System is a real-time, AI-based event selector designed to ingest high-speed raw data from front-end analog-to-digital converter(ADC) digitizers, perform low-latency inference using parallel CNN Accelerator Cores, and trigger the readout of high-fidelity waveform data.

The architecture employs a Split-Path Strategy to decouple data storage from analysis. Incoming data is bifurcated into a Storage Path (High-Fidelity Ground Truth) and an Inference Path (Downsampled/Selected Channels). A central PicoRV32 soft-core orchestrates a pool of parallel CNN Accelerators and a composite Hi-Lo trigger via a dedicated DMA engine. Use a composite hi-lo trigger with a simplified CNN trigger as the trigger scheme for maintenance and emergency situations. While allowing simple task logic to be controlled via the host computer, system logic can also be rewritten by writing binary files.

1.1 Top-Level Block Diagram

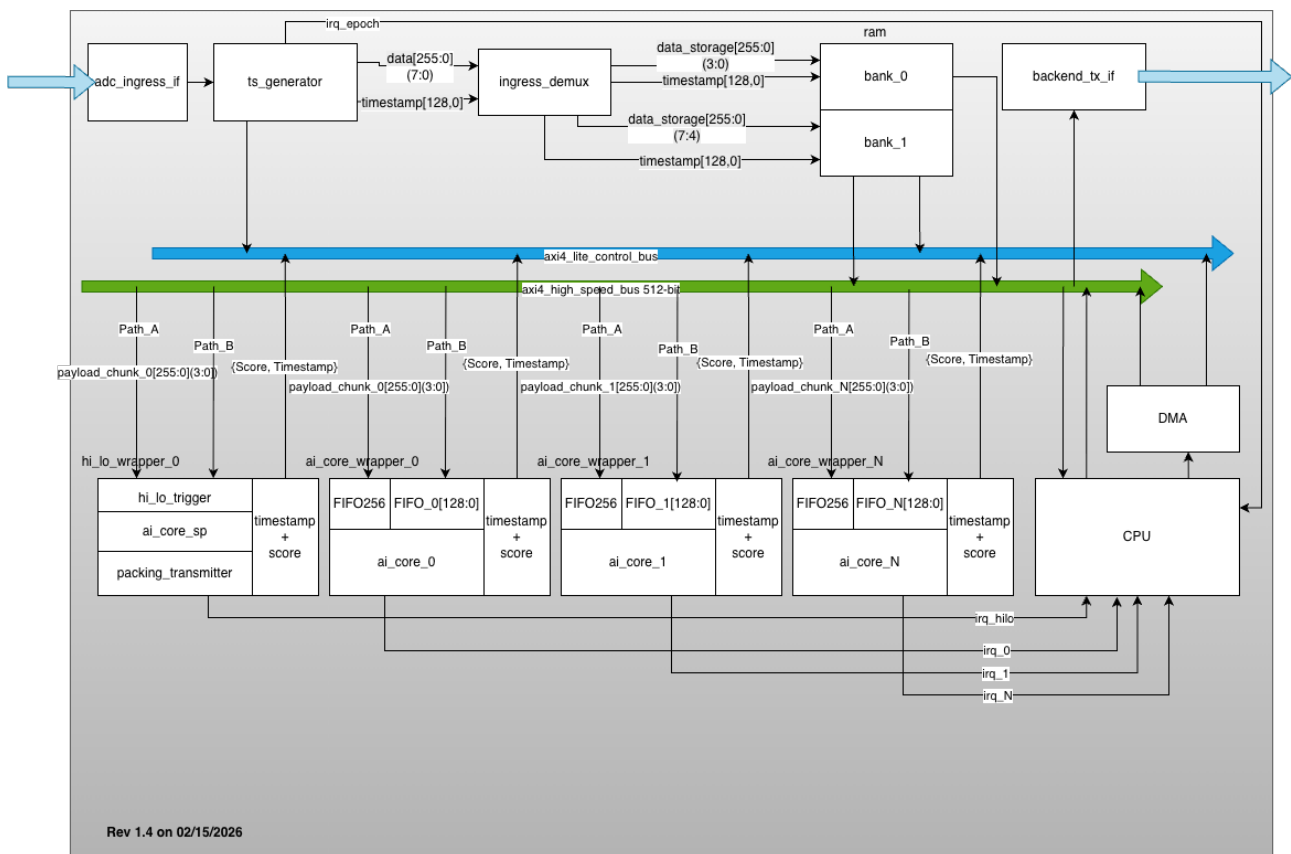


Figure 1: Top Level Architecture of the AI Trigger System

1.2 Key Subsystems

Data Ingest (`ingress_subsys`)

Handles the reception and initial timestamping of raw data.

- `adc_ingress_if`: Receives raw `adc64_to_uart` packets from the front-end. It connects directly to the raw data stream (from a 1GHz ADC digitizer, 1 sample * 8 channels per nanosecond). The front-end module sends 64 samples for each 8 channels at a time (*ADC Data Package Size*, which can be configured), with each sample having a 12-bit width (*Sample-Bitwidth*, which can be configured). This module will bundle chunks according to the specified *Chunk Size*, which can be configured. For example, if the



chunk size is set to 256, it will merge four data packets from the front-end ADC module into a single chunk, as the timestamps used for numbering are based on the chunk size.

- **ts_generator:** Implements a Two-Tier Timing engine ($T_{short} \times T_{epoch}$) to tag incoming data. This mechanism balances the system's limited resources with the need for long-term sample tracking. All chunks in the system—the smallest trigger units—are tracked via T_{short} . For more information, see section 3.3: Timing and Wrapper Logic.
- **ingress_demux:** Splits the 8-channel input stream into two physical memory banks to optimize bandwidth usage. The four channels specified in the program (e.g., 0-3) enter `bank_0`, while the remaining four channels enter `bank_1`. Within each chunk, only the four channels in `bank_0` participate in inference. All eight channels enter the backend system after triggering and are acquired by the host computer. In this document, unless otherwise specified, the "lower four digits", or "0-3 digits" refer by default to the selected four channels.

Storage and Inference Plane

The core memory architecture supports simultaneous read and write operations.

- **bank_0 (The Hot Bank):** Stores selected channels 0-3 (12-bit). Acts as the primary source for both the *Compute Cluster* (continuous high-speed access) and backend readout (triggered access).
- **bank_1 (The Cold Bank):** Stores channels 4-7 (12-bit). Accessed only by the backend interface when an event is triggered.
- **Bank Capacity:** Both banks are circular buffers sized to hold 128 full chunks. This *Bank Capacity* shall be configurable to balance resources and demand.

Control Plane (AXI4-Lite)

Managed by a PicoRV32 RISC-V soft-core CPU. It is responsible for high-level task scheduling, interrupt aggregation via `SR_CORE_DONE`, and managing system state transitions.

Data Plane (AXI4 High-Speed)

A 512-bit wide high-bandwidth backbone (`axi4_high_speed_bus`) connecting `ram_bank_0`, DMA, and the compute cluster. It ensures a transfer rate of < 30 cycles per chunk to satisfy real-time constraints.

Compute Cluster

Operates as a hybrid array consisting of parallel `ai_core_wrapper` units (Core 0 to Core N) and traditional Hi-Lo Trigger units. These units perform concurrent inference and magnitude comparison to identify physics events. Each unit manages synchronization between the raw data stream and the side-channel timestamp, ensuring results remain correctly tagged despite pipeline latency.

Watchdog System

An internal hardware monitor located within the cluster that observes the output timing and content of each individual trigger unit. It identifies core-level stalls using a configurable *Hang Time* and initiates the recovery protocol to ensure no *unprocessed* chunks are lost.

CPU

The PicoRV32 CPU serves as the system's central orchestrator, implementing control logic and interacting with the host computer.

- Manages task scheduling by monitoring the `ts_generator` and dispatching idle compute units via a "First-Available" scheduler.
- Responsible for Trigger Decision Logic, re-ordering asynchronous results and comparing them against the *Trigger Threshold*.
- Handles fault recovery by responding to Watchdog alarms and re-queuing failed tasks.
- Manages state machine transitions between *Idle*, *Run*, and *Passthrough* modes.



DMA

A dedicated hardware engine designed for autonomous bulk data movement across the 512-bit AXI4 High-Speed Bus.

- Minimizes CPU overhead by handling the direct transfer of chunks from `bank_0` to the hybrid Compute Cluster.
- Supports a "Just-In-Time" refill mechanism, automatically bursting new data into unit FIFOs when levels are low.
- Executes the final readout of high-fidelity waveforms from both memory banks to the backend interface upon a positive trigger decision.

2 Clocking and Synchronization

To maintain the strict 256 ns throughput requirement while ensuring timing closure across the FPGA fabric, the system utilizes multiple asynchronous and synchronous clock domains managed by on-chip Phase-Locked Loops (PLLs).

3 Functional Description

3.1 Split-Bank Memory Architecture

To maximize memory bandwidth and eliminate redundant storage, the system employs a Banked Memory Strategy. Instead of separate buffers for storage and inference, the 8-channel signal is split across two parallel RAM banks. The lower four channels serve both storage and triggering functions, while the upper four channels are used solely for storage.

3.1.1 Ingress Signal Splitting (`ingress_demux`)

The demultiplexer separates the assembled 8-channel chunk into two parallel write paths:

- **Path A (Low Channels):** `adc_data[3:0]` are written to `bank_0` at address T_{short} .
- **Path B (High Channels):** `adc_data[7:4]` are written to `bank_1` at address T_{short} .

3.1.2 Banked Memory Roles

`bank_0` (Dual-Use)

- **Inference Task:** The DMA reads 256 samples from this bank to feed the *Compute Cluster*. Since this bank is only 4 channels wide, the transfer is at least 50% faster than a full-width read.
- **Trigger Task:** Upon a triggered event, the DMA reads this bank to retrieve the lower half of the waveform.

`bank_1` (Storage Only)

- **Trigger Task:** Accessed only when a trigger decision is positive. The DMA reads this bank to retrieve the upper half of the waveform.

3.1.3 Addressing and Circularity

Both banks share the same write pointer (T_{short}) driven by the `ts_generator`.

- **Write Address:** $Base + (T_{short} \times ChunkSize)$.
- **Overwrite:** The buffer wraps every 128 chunks (*Bank Capacity*). The system relies on `SR_WRITE_PTR` to ensure the CPU does not read data that is actively being overwritten. The Bank Capacity shall be configurable to balance resources and demand.



3.2 Task Orchestration

3.2.1 Dynamic “First-Available” Scheduler

The PicoRV32 firmware implements a high-frequency, low-latency orchestration loop designed to saturate the hybrid compute cluster and maintain the 256 ns per chunk throughput requirement. The scheduling logic follows a five-stage execution pattern to accommodate both AI and Hi-Lo units, as well as automated fault recovery:

1. **Poll (Sync and Identification):** The CPU continuously monitors the `ts_generator` via the `SR_WRITE_PTR` register to identify the most recent Timestamp ID (T) committed to memory. The firmware tracks the *Lag* to prioritize the allocation of the earliest pending chunk. This ensures temporal causality and prevents buffer overflow in the circular `bank_0`.
2. **Arbitration (Resource Selection):** The CPU performs a bitwise scan of the `SR_CORE_READY` global bitmask. Each bit i in this register indicates the operational status of a unit in the cluster (inclusive of CNN and Hi-Lo wrappers). The scheduler identifies the first available bit set to 1, designating that idle unit as the target for the next task.

The priority of the emergency trigger section containing hi-lo is defined as the lowest. It is only utilized when processing speed is identified as not falling below a certain threshold. Thus, even if data that has not yet entered the trigger process remains in memory, as long as the entire system can withstand the data flow intensity, it will not all enter the low-level trigger (i.e., the hi-lo trigger computation unit containing simple CNN triggers) at once.

3. **Dispatch (DMA Configuration):** Once a compute unit i and timestamp T are paired, the CPU populates the DMA control registers:
 - `CR_DMA_SRC`: Calculated as $Base_Address + (T \times ChunkSize)$, pointing to the 4-channel data in `bank_0`.
 - `CR_DMA_DEST`: The memory-mapped slave address for the target unit’s input FIFO.
 - `CR_DMA_LEN`: Total bytes defined by the *Sample-Bitwidth* and RTL-fixed *Chunk Size*.
 - `CR_DMA_CTRL`: Set to `0x1` (START) to initiate the high-speed burst.
4. **Monitor (Watchdog Integration):** As the DMA engine takes mastery of the 512-bit bus to execute the transfer (budget ≤ 30 cycles), the internal watchdog monitors the unit’s latency. If the unit exceeds the *Hang Time*, the watchdog asserts a `WD_ALARM`. The scheduler is interrupted to handle the recovery protocol, marking the chunk as *unprocessed* for re-submission.
5. **Repeat (Asynchronous Handshake):** Under normal operation, the data movement is decoupled from the Control Plane. The PicoRV32 immediately increments its `next_expected_ts` pointer and loops back to Step 1. This pipelining allows the CPU to manage up to 8 parallel units simultaneously without incurring processing stalls.

3.3 Timing and Wrapper Logic

The system employs a multi-stage timing architecture to reconcile the limited on-chip memory depth with the need for long-term time tracking.

3.3.1 AI Accelerator Wrapper (`ai_core_wrapper`)

The wrapper acts as an isolation layer, decoupling the high-speed data transport from the deterministic latency of the Neural Network IP.

- **Input Burst Buffering (FIFO256):** Receives data from the 512-bit bus (fill time ≈ 30 cycles) and drains into the IP Core (process time > 1000 cycles).



- **“Just-In-Time” Refill Mechanism:** To eliminate dead time, when $\text{FIFO_LEVEL} < 4$, the Wrapper asserts a hardware `dma_req`. The DMA immediately bursts the next chunk from `bank_0`, overlapping transfer with processing.
- **Side-Channel Timestamp Synchronization:** The wrapper pushes the Input Timestamp (T_{short}) into a parallel Delay FIFO. When the AI core, i.e., CNN, outputs a valid Score, the Delay FIFO pops the corresponding Timestamp to maintain strict temporal causality.

3.3.2 Two-Tier Timing Engine

Two-Tier Timing is designed to conserve resources by avoiding the tagging of each chunk. Since a large number of chunks may be processed simultaneously within the system, and varying return times occur in systems handling high parallelism and multiple concurrent triggers, it is necessary to number the chunks.

- T_{short} : This is a system-wide timestamp (second-level timestamp) used to assign a relative arrival time to each chunk. Its maximum value is 128. It is immediately assigned to a chunk when ADC data enters the trigger system and remains unchanged throughout the inference and data flow. Only after the backend system receives the chunk is it reused. This numbering is incremented cyclically. The relative sequence number output within the same cycle represents the relative order of chunk arrival. If a chunk triggers, all its channels are output with this timestamp; if it does not trigger, no output occurs.
- T_{epoch} : This is the primary timestamp used to mark the number of T_{short} cycles. Its maximum value is 512. It spans the entire data preprocessing and transmission system. Its sequence number increments by 1 each time a T_{short} cycle is used. Unlike T_{short} , its sequence number always increments by 1 regardless of whether an output was triggered in the current cycle. This timestamp enables monitoring of the timing system’s proper operation during output and tracking of data’s relative timing. The backend can quickly obtain real-world time on the host machine using simple algorithms and network connectivity.

3.4 Trigger and Re-Ordering Logic

3.4.1 Software Re-Order Buffer

Due to the highly parallel and asynchronous nature of the compute cluster, inference results from the AI accelerator cores may complete out of temporal sequence. This occurs when varying signal complexities lead to non-deterministic processing latencies across different cores (e.g., the inference result for Timestamp $T = 5$ may be available before $T = 4$). Additionally, the occurrence of special circumstances, including potentially diverse triggering mechanisms and various types of trigger cores, as well as states such as maintenance mode, emergency mode, and debug mode, may also result in asynchronous chunk numbering within the compute cluster’s throughput.

To maintain a strictly causal, time-ordered data stream for the backend interface and the host computer, the PicoRV32 firmware implements a Software Re-Order Buffer (ROB). This mechanism effectively decouples the parallel execution of the Neural Network IP from the sequential requirements of the physics data stream.

- **Result Ingestion:** Upon the assertion of the `SR_CORE_DONE` interrupt aggregator, the CPU identifies the specific core that has completed its task. The firmware then performs a memory-mapped read of the corresponding core’s `SR_CORE_RESULT` register to retrieve the 8-bit *Score* (*Score Bit-width* shall be configurable) and the 13-bit *Timestamp* (T_{short}) associated with the processed chunk.
- **Non-Blocking Buffering:** Rather than triggering an immediate data readout, the CPU populates a software-defined staging array: `pending_results[Timestamp] = Score`. This allows results from faster-processing cores to be held in situ without stalling the global dispatch scheduler or affecting the “First-Available” core selection logic.



- **Sequential Ordering:** The firmware maintains a local state pointer, `next_expected_ts`, which tracks the next required chunk in the T_{short} cycle (0–127). The CPU executes a blocking check on this pointer:
 - If `pending_results[next_expected_ts]` contains a valid score, the CPU proceeds to execute the *Trigger Decision Logic*.
 - If the result is not yet available, the CPU waits, acting as a synchronization barrier to ensure that high-fidelity waveform data is never transmitted to the backend out of order.
- **Pointer Advancement and Cleanup:** Following a successful trigger decision (regardless of a positive or negative result), the CPU clears the relevant entry in the `pending_results` array to prepare for the next circular buffer cycle. The `next_expected_ts` pointer is then incremented using modulo-128 arithmetic to maintain alignment with the circular ingress buffer.

3.4.2 Thresholding and Backend Data Move

Upon the synchronization of the `next_expected_ts` pointer with a valid entry in the Software Re-Order Buffer, the PicoRV32 firmware initiates the Trigger Decision Logic and the subsequent high-fidelity data extraction sequence. This stage is critical for ensuring that only events of physics interest occupy the downstream telemetry bandwidth.

Threshold Evaluation The extracted 8-bit *Score* is compared against the `TRIGGER_THRESHOLD` register, which is globally configurable via the AXI4-Lite control plane. This comparison determines the operational branch of the readout state machine.

Positive Trigger Decision ($Score \geq Threshold$)

The system initiates a full-resolution readout of the 8-channel event corresponding to the current timestamp T . To maintain the integrity of the split-path architecture, the readout is executed in two atomic DMA transactions:

1. **Transfer 1 (Primary Channels):** The CPU configures the DMA to extract 256 samples (12-bit width) from the "Hot Bank" (`bank_0`) at the memory offset calculated from T . This data is pushed to the `backend_tx_if` for packetization.
2. **Transfer 2 (Secondary Channels):** Immediately following the first completion, the CPU dispatches the DMA to extract the remaining 256 samples from the "Cold Bank" (`bank_1`).

Outcome: The backend interface reconstructs the complete, 8-channel high-fidelity waveform for long-term storage and analysis.

Negative Trigger Decision ($Score < Threshold$)

In the absence of a significant physics signature, no DMA dispatch is triggered. The ingress data residing in the circular buffer is flagged as disposable and will be overwritten by the `adc_ingress_if` after exactly 128 chunk cycles.

Pointer Management Following the trigger resolution, the `next_expected_ts` pointer is incremented using a modulo-128 operation. This ensures the Software Re-Order Buffer remains synchronized with the hardware write pointers and the `ts_generator` epoch heartbeat.

4 CPU

The control and orchestration layer of the ARIANNA AI Trigger System is centered upon the PicoRV32, a high-performance, size-optimized RISC-V CPU. This chapter provides an exhaustive technical breakdown of the CPU's architecture, its specific configuration for this project, and the custom ISA extensions utilized for real-time interrupt handling and task scheduling.

The specific CPU is configurable. PicoRV32 serves as an industry benchmark and is used here as a prototype.



4.1 Architectural Philosophy and Design Goals

The PicoRV32 is engineered to serve as an auxiliary processor within FPGA designs, emphasizing a high maximum operating frequency (f_{max}) and minimal logic utilization. By achieving an f_{max} of 250–450 MHz on 7-series Xilinx hardware, the core operates within the primary clock domain of the high-speed digitizers, eliminating the latency penalties associated with asynchronous clock-domain crossing (CDC).

The core utilizes a size-optimized pipeline that can be configured to favor either area or performance, making it ideal for the limited resource environment of the ARIANNA trigger boards.

4.2 Instruction Set Architecture (ISA) and Configuration

The ARIANNA implementation of the PicoRV32 is configured for the **RV32IMC** ISA. This selection provides a balance between computational power and code density:

- **RV32I (Base Integer):** Implements the standard 32-bit integer instruction set, including 32 general-purpose registers (x0 to x31).
- **M-Extension (Multiplication/Division):** Includes high-performance hardware for MUL, DIV, and REM operations. These are implemented via the internal Pico Co-Processor Interface (PCPI) to maintain timing closure at high frequencies.
- **C-Extension (Compressed):** Supports 16-bit instruction encodings, reducing the footprint of the control firmware in the limited on-chip instruction memory.

The core is further optimized via specific Verilog parameters to meet the 256 ns throughput requirement:

- `ENABLE_REGS_DUALPORT = 1`: Facilitates simultaneous register reads, reducing Cycles Per Instruction (CPI) for ALU operations.
- `BARREL_SHIFTER = 1`: Ensures shift operations are completed in a single cycle, preventing stalls during data packet parsing.
- `TWO_CYCLE_ALU = 1`: Adds a pipeline stage to the ALU path, allowing the design to meet timing at high frequencies by relaxing the combinatorial path.
- `CATCH_MISALIGN = 1`: Enables hardware detection of misaligned memory accesses, triggering IRQ 2 upon fault.

4.3 Performance Metrics

The PicoRV32 is designed for size efficiency; however, its performance is sufficient for orchestrating the hybrid compute cluster. The average CPI for the ARIANNA firmware is approximately 4.10 when utilized with the `BARREL_SHIFTER` and dual-port register file.

Table 1: Instruction Latency and CPI Performance

Instruction Type	CPI (Dual-Port)	Description
jal (Direct Jump)	3	Unconditional control flow change.
ALU Register + Immediate	3	Single-cycle arithmetic with immediate.
ALU Register + Register	3	Concurrent register read and execution.
Memory Load	5	Data retrieval from the memory interface.
Memory Store	5	Data commitment to register or RAM.
Taken Branch	5	Conditional jump execution.
Hardware MUL	40	PCPI-based multiplication.



4.4 Native and System Interfaces

The core utilized in this specification is the `picorv32_axi` variant . It translates the native internal memory interface into a standard AXI4-Lite Master interface.

- **AXI4-Lite Master:** Used to interact with the Global Control Registers and Core-Specific Registers defined in the following chapters .
- **Pico Co-Processor Interface (PCPI):** Used for non-branching custom hardware accelerators. In the ARIANNA design, this interface facilitates the integration of high-speed multiplication and division cores.
- **Look-Ahead Interface:** Provides early signals (`mem_la_read`, `mem_la_write`, and `mem_la_addr`) to the memory controller one cycle before the transaction begins.

4.5 Custom Interrupt Handling and IRQ Management

The PicoRV32 does not utilize the standard RISC-V Privileged ISA for interrupts, opting instead for a size-optimized custom interrupt controller. This mechanism is critical for the **Watchdog-Driven Fault Recovery** logic.

4.5.1 Interrupt State Registers

The core implements four specialized 32-bit registers (`q0` to `q3`) for context management during an IRQ event:

- `q0`: Automatically stores the return address of the interrupted instruction.
- `q1`: Automatically captures a bitmask of all pending IRQs (e.g., Bit 31 for `WD_ALARM`).
- `q2--q3`: General-purpose "shadow" registers used by the interrupt handler for temporary storage.

4.5.2 Specialized IRQ Instructions

A dedicated set of custom instructions is available to firmware for managing real-time exceptions. All instructions are encoded under the `custom0` opcode:

- `getq rd, qs`: Copies a value from a q-register to a general-purpose register.
- `setq qd, rs`: Copies a value from a general-purpose register to a q-register.
- `maskirq rd, rs`: Writes a new bitmask to the interrupt controller and reads the previous mask, allowing selective enabling of interrupts.
- `waitirq rd`: Suspends CPU execution until an interrupt becomes pending, reducing power and bus overhead.
- `retirq`: Atomically restores the program counter from `q0` and re-enables the global interrupt state.
- `timer`: Resets the built-in countdown timer and returns the old value.

4.6 Built-in Interrupt Sources

The internal controller reserves IRQs 0–2 for hardware-level exceptions :

- **IRQ 0:** Timer interrupt, triggered when the internal counter transitions from 1 to 0.
- **IRQ 1:** EBREAK/ECALL trap or Illegal Instruction exception .
- **IRQ 2:** Bus Error, typically triggered by misaligned memory accesses.

All other IRQ lines (3–31) are mapped to external signals, including the **Watchdog Alarm** and the **Core Done** interrupt aggregator.



5 Memory Map and Register Description

The system utilizes a memory-mapped I/O architecture on the AXI4-Lite control bus. The PicoRV32 soft-core acts as the system master, orchestrating the DMA engine, monitoring the high-speed data ingress, and managing the parallel compute cluster through the following register set.

5.1 Global Control Registers

The Global Control Registers manage system-wide orchestration, DMA transfers, and high-level configuration. While *Chunk Size* is a fixed RTL constant, parameters such as *Sample-Bitwidth* and Watchdog timing remain software-configurable.

Table 2: Global Control Registers Map

Offset	Name	Access	Width	Description
0x00	CR_DMA_CTRL	R/W	32	Bit 0: START. Dispatch a DMA burst transfer. Bit 1: IRQ_CLR. Clear DMA completion interrupt. Bit 2: SOFT_RESET. Flush DMA FIFOs and reset state machine.
0x04	CR_DMA_SRC	R/W	32	DMA Source Address. Memory offset in Bank 0 or Bank 1.
0x08	CR_DMA_DEST	R/W	32	DMA Destination Address. Target Compute Cluster input FIFO.
0x0C	CR_DMA_LEN	R/W	32	Transfer Length. Total bytes per chunk (based on bitwidth).
0x10	CR_TRIGGER_THRES	R/W	32	[7:0]: Trigger Threshold. Score required for AI-based readout.
0x14	CR_CONFIG_GEN	R/W	32	[7:0]: Sample-Bitwidth. ADC resolution (Default: 12). [15:8]: Bank Capacity. Buffer depth (Default: 128 chunks).
0x18	SR_WRITE_PTR	R	32	[6:0]: Current T_{short}. Hardware ingress write pointer (0–127).
0x1C	SR_CORE_READY	R	32	Ready Bitmask. Bit $i = 1$ implies Core i is IDLE.
0x20	SR_CORE_DONE	R/W1C	32	Interrupt Aggregator. Bit $i = 1$ indicates Core i finished. Bit 31: WD_ALARM. Watchdog detected a core hang/anomaly.
0x24	CR_WATCHDOG_CFG	R/W	32	[15:0]: Hang Time. Max clock cycles before core timeout. [31]: WD_EN. Global watchdog enable.
0x2C	CR_EMERGENCY	R/W	32	[7:0]: Batch Size. For emergency force-triggers. [31]: Force Active. Manual emergency mode override.



5.2 Core-Specific Registers

These registers are repeated for each unit in the hybrid compute cluster (AI and Hi-Lo). The Base Address for Core i is calculated as: $\text{Global_Base} + 0x100 + (i * 0x20)$.

Table 3: Core-Specific Registers (Offset from Core Base)

Offset	Name	Access	Width	Description
0x00	CR_CORE_CTRL	R/W	32	Bit 0: START. Manually trigger inference/comparison. Bit 1: IRQ_CLR. Acknowledge local interrupt. Bit 7: RESET. Watchdog/CPU targeted unit reset.
0x04	SR_CORE_RESULT	R	32	[7:0]: Score. 8-bit confidence or Hi-Lo magnitude. [14:8]: Timestamp. Associated 7-bit T_{short} ID (0–127).
0x08	SR_CORE_STATUS	R	32	Bit 0: BUSY. Unit is currently processing. Bit 1: DONE. Valid result waiting in buffer. Bit 31: HANG. Watchdog flagged this unit as stalled.

5.3 Hardware-Software Handshake Sequence

The following sequence formalizes the interaction between the PicoRV32 Control Plane and the hybrid Data Plane, including automated watchdog monitoring:

- Synchronization:** The CPU polls `SR_WRITE_PTR` to ensure the target T_{short} chunk is committed to RAM.
- Arbitration:** The CPU reads `SR_CORE_READY` to select an available compute unit (AI or Hi-Lo).
- Transfer:** The CPU configures the DMA source/destination and asserts `CR_DMA_CTRL[0]`.
- Monitoring:** The internal watchdog tracks output timing based on `CR_WATCHDOG_CFG`. If a unit exceeds *Hang Time*, `WD_ALARM` is asserted.
- Recovery:** Upon a watchdog alarm, the CPU identifies the stalled unit via `SR_CORE_STATUS[31]`, executes a targeted reset, and re-submits the “unprocessed” chunk ID to the cluster.
- Collection:** During normal operation, the CPU retrieves results from `SR_CORE_RESULT` upon a `SR_CORE_DONE` interrupt.
- Re-Order:** Results are stored in the Software Re-Order Buffer (ROB) at index `Timestamp`.
- Finalization:** When `next_expected_ts` points to a valid ROB entry, the CPU executes the Trigger Decision against `CR_TRIGGER_THRES`.

6 Timing and Performance Constraints

To ensure deterministic event selection in high-energy physics environments, the system must adhere to the following timing and throughput constraints.

- Throughput Requirement:** The system must maintain a steady-state processing rate of one chunk every 256 ns. This requirement is driven by the 1 GHz ADC sampling rate and the fixed *Chunk Size* of 256 samples.
- Bus Bandwidth and Data Plane:**



- **Bus Configuration:** The `axi4_high_speed_bus` must be configured with a data width of **512 bits** to provide sufficient headroom for parallel core access.
- **Transfer Budget:** At 512-bit width, a single 4-channel inference chunk (12,288 bits total) must be transferred from `bank_0` to a compute unit in < 30 clock cycles.
- **Refill Latency:** The DMA arbitration and bus burst must complete within the processing window of the final 4 samples of a previous chunk's inference to ensure a "Just-In-Time" refill and prevent core starvation.
- **Cluster Concurrency and Contention:**
 - The system supports N parallel compute units. Based on a 512-bit bus backbone, the DMA bandwidth is optimized to saturate up to ≈ 8 cores simultaneously.
 - Beyond 8 cores, bus contention and AXI4-Lite arbitration latency may impact the 256 ns throughput requirement.
- **Watchdog and Recovery Timing:**
 - **Hang Time:** The configurable *Hang Time* must be set to a value greater than the deterministic latency of the AI IP (typically > 1000 cycles) but low enough to allow the CPU to re-queue *unprocessed* chunks before they are overwritten in the circular buffer.
 - **Buffer Headroom:** The *Bank Capacity* (128 chunks) provides a safety margin of approximately $32.7 \mu\text{s}$. Fault recovery and chunk re-submission must complete within this window to avoid data loss.

7 Operational Modes

The system implements a hierarchical state machine managed by the PicoRV32 Control Plane. Operational logic is executed by the Compute Cluster, which now operates as a hybrid array consisting of parallel AI Accelerator Cores and traditional Hi-Lo Trigger units.

7.1 Idle Mode

- **Status:** System is powered and responsive to AXI4-Lite commands; data processing is halted.
- **Behavior:** The `ingress_subsys` ignores incoming ADC streams. All DMA engines are disabled, and compute cluster interrupts are masked.
- **Use Case:** System initialization, and configuring the `TRIGGER_THRESHOLD` or Hi-Lo parameters.

7.2 Run Mode

- **Status:** Fully active event selection utilizing the complete compute cluster.
- **Behavior:** The DMA engine automatically fetches chunks from Bank 0. The CPU orchestrates task distribution across both AI and Hi-Lo trigger units. The "Re-Order & Threshold" loop ensures that only events meeting the combined trigger criteria are transmitted to the backend.
- **Use Case:** Standard data acquisition for physics experiments.

7.3 Passthrough Mode

- **Status:** Compute cluster inference is bypassed to collect unbiased calibration data.
- **Behavior:** The CPU forces a positive trigger decision periodically (e.g., every k -th chunk). Raw, unbiased waveforms from both memory banks are packetized and sent to the host computer.



7.4 Watchdog and Fault Recovery

To ensure the reliability of the high-concurrency architecture, a dedicated Watchdog system is integrated within the compute cluster. This hardware monitor observes the output timing and content of each individual trigger unit (AI and Hi-Lo).

- **Anomaly Detection:** The watchdog monitors the latency of each core. If a specific trigger remains inactive for a configurable *Hang Time* while the rest of the cluster continues to process chunks, it identifies a core-level stall.
- **Fault Signaling:** Upon detecting a stall or malformed output, the watchdog asserts an alarm interrupt to the PicoRV32 CPU.
- **Recovery Protocol:** The CPU executes a targeted reset of the failed compute unit. The specific chunk in Bank 0 associated with the failed inference is flagged as “unprocessed.”
- **Data Integrity:** Flagged chunks are re-queued for entry into the compute cluster, ensuring that no potential physics events are lost due to transient hardware hangs.

8 System Initialization and Error Recovery

8.1 Cold Boot Sequence

Upon power-up or a global Hardware Reset (r_{st_n}), the PicoRV32 firmware must execute a deterministic initialization sequence to transition the system from an undefined state to a stable **Idle Mode**.

1. **Global Aggregator Reset:** Write $0xFFFFFFFF$ to `SR_CORE_DONE` to clear all completion interrupts and the `WD_ALARM` (Bit 31).
2. **DMA Quiescence:** Explicitly disable the DMA engine by writing $0x0$ to `CR_DMA_CTRL` to prevent unintended memory access during boot.
3. **Watchdog Initialization:** Write $0x0$ to `CR_WATCHDOG_CFG` to ensure the watchdog is disabled during the sanity check phase.
4. **Hybrid Cluster Sanity Check:** Iterate through all compute units (Cores $0 \dots N$):
 - Assert the `RESET` bit in `CR_CORE_CTRL`.
 - Poll `SR_CORE_STATUS` until `BUSY` is low and `READY` is high.
5. **Software State Reset:** Zero-initialize the `pending_results` array (Software ROB) and reset the `next_expected_ts` pointer to 0.

8.2 Run Start Procedure

To transition the system into active data-taking (**Run Mode**), the following configuration steps must be performed:

1. **Parameter Loading:** Write the `Sample-Bitwidth` to `CR_CONFIG_GEN` and the experiment-specific `CR_TRIGGER_THRES`.
2. **Watchdog Arming:** Write the desired cycle count to the *Hang Time* field and set the `WD_EN` bit in `CR_WATCHDOG_CFG`.
3. **Timing Synchronization:** Enable the `ts_generator`. The CPU must execute a blocking wait for the first `irq_epoch` interrupt to ensure temporal alignment between physical memory and the Software ROB.
4. **Scheduler Entry:** Enable global interrupts and enter the task arbitration loop.



8.3 Watchdog-Driven Fault Recovery

The hardware Watchdog module acts as the primary recovery agent for system lockups. The CPU behavior is triggered by and dependent on the signals provided by the Watchdog module.

1. **Hardware-Initiated Reset:** When a core exceeds the *Hang Time*, the Watchdog hardware autonomously asserts the core-local RESET line, sets the HANG bit in SR_CORE_STATUS, and triggers the WD_ALARM in the global aggregator.
2. **Data Flagging:** The Watchdog hardware internally prevents the completion of the stalled task and flags the associated chunk in Bank 0 as "unprocessed" for the Control Plane.
3. **CPU Alarm Response:** Upon receiving the WD_ALARM interrupt, the CPU identifies the failed unit and retrieves the stalled *Timestamp* ID from the unit's SR_CORE_RESULT register.
4. **Task Re-submission:** To maintain data integrity, the CPU re-submits the "unprocessed" chunk ID to the next available compute unit in the cluster.
5. **Sequence Resumption:** The CPU writes a 1 to clear the WD_ALARM bit and resumes the main Scheduler Loop.



A Appendix

A.1 Term and Parameter Definitions

The following terms and configurable parameters are used throughout the specification. These parameters are listed in *italics* within the technical prose.

Table 4: System Terms and Definitions

Term	Definition
ADC	Analog-to-Digital Converter. The front-end hardware component responsible for digitizing analog waveform signals into discrete digital samples.
ADC Data Package Size	A configurable parameter defining the number of samples sent per channel per transfer from the front-end module.
Bank Capacity	A configurable parameter defining the number of full chunks held within the circular memory buffers (Default: 128).
Chunk	Payload Chunk. The fundamental unit of data processing. One chunk consists of 256 contiguous time samples across all 8 channels. The size is fixed at 12 bits per sample per channel.
Chunk Size	A configurable parameter (fixed in RTL for throughput) defining the number of samples merged from the front-end ADC packets into a single processing unit.
Control Plane Domain (f_{cpu})	The clock domain (250–450 MHz) driving the PicoRV32 CPU and the AXI4-Lite control bus.
Data Plane Domain (f_{data})	The clock domain (200–250 MHz) driving the high-bandwidth AXI4 High-Speed Bus and DMA engines.
DMA	Direct Memory Access. A hardware engine moving bulk data between memory banks and the compute cluster via the 512-bit High-Speed Bus without CPU intervention.
Emergency Mode	A fail-safe operational state triggered when processing lag drops below the safety headroom ($Gap < 10$). The system automatically triggers (force-accepts) chunks to clear the backlog.
Epoch (T_{epoch})	A coarse-grained 9-bit counter used to track time over longer durations. Increments every time T_{short} wraps (every 128 chunks). Used for backend time reconstruction.
Hang Time	A configurable cycle count threshold used by the hardware watchdog to detect core-level stalls or processing anomalies.
Heartbeat	A synchronization packet (Type B) containing the current T_{epoch} , transmitted every 128 chunks to keep the Backend DAQ synchronized with the FPGA clock.
High-Speed Sampling Domain (f_{adc})	The 1 GHz clock domain used for the initial ingestion of raw digitizer data.
Lag	The distance (in chunks) between the current Write Pointer (Ingress) and the Read Pointer (Inference). Formula: $(SR_WRITE_PTR - next_expected_ts) \% 128$.
PicoRV32	The soft-core RISC-V processor acting as the system Control Plane. It manages task scheduling, interrupts, and register configuration via AXI4-Lite.

Continued on next page



Table 4 – continued from previous page

Term	Definition
q0–q3	Specialized 32-bit registers within the PicoRV32 used for custom interrupt state management and context saving.
<i>Sample-Bitwidth</i>	A configurable parameter defining the precision of the digitized waveform samples (Default: 12-bit).
Score	An 8-bit confidence value (0–255) generated by the CNN Accelerator Core, indicating the likelihood that a chunk contains a physics event.
<i>Score Bit-width</i>	A configurable parameter defining the resolution of the confidence output from the compute unit.
Timestamp (T_{short})	A fine-grained 7-bit cyclic counter (0–127) tagging every chunk. Serves as the physical memory address index for the circular buffers.
Trigger Threshold	A configurable parameter. If a chunk's <i>Score</i> exceeds this value, the system triggers a full high-fidelity readout.
<i>unprocessed</i>	A status flag assigned to data chunks that failed to complete inference due to a watchdog reset, requiring re-submission to the compute cluster.
WD_ALARM	An interrupt signal (Bit 31 of the aggregator) asserted by the hardware watchdog when a unit in the compute cluster exceeds the <i>Hang Time</i> .